ARQUITETURA CUDA E O OVERHEAD DO WINDOWS 11

Benchmark de latência de tokens — llm-ada-project

`timed_generate` / `TokenTimer` — outubro de 2026

ROTEIRO

1. Arquitetura CUDA: abstrações e como a GPU executa (base: CMU 15-779, aulas 3–4)
2. Por que o decode batch=1 é latency-bound e bandwidth-bound
3. Launch, sincronização, streamer, clocks, tensor cores e 4-bit no nosso benchmark
4. Windows 11: de onde vem o overhead e como calculá-lo
5. Protocolo de medição e checklist

Nada aqui foi executado em GPU: esta máquina não tem CUDA. Códigos e exemplos numéricos são propostas para rodar no Colab T4 / PC Windows.

1. ARQUITETURA CUDA

ABSTRAÇÕES CUDA: GRID, BLOCO, THREAD

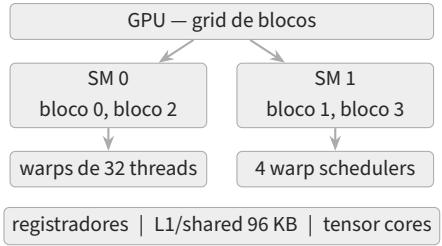
- Host (CPU) lança um kernel; o device (GPU) executa
- Kernel = grid de blocos; bloco = até 1024 threads
- Cada thread acha seus dados por `blockIdx`, `threadIdx`
- Blocos são independentes: a GPU os executa em qualquer ordem
- Lançamento é assíncrono: a CPU segue em frente

```
__global__ void matrixAdd(float A[Ny][Nx],
    float B[Ny][Nx], float C[Ny][Nx]) {
  int i = blockIdx.x*blockDim.x + threadIdx.x;
  int j = blockIdx.y*blockDim.y + threadIdx.y;
  C[j][i] = A[j][i] + B[j][i];
}
// host
dim3 threadsPerBlock(4, 3);
dim3 numBlocks(Nx/4, Ny/3);
matrixAdd<<<numBlocks, threadsPerBlock>>>(A,B,C);
```

Exemplo adaptado de CMU 15-779 (Z. Jia, 2025).

DO BLOCO AO WARP: COMO A GPU EXECUTA

- O escalonador mapeia blocos em SMs enquanto houver recursos (registradores, shared, slots de warp)
- Bloco é quebrado em warps de 32 threads (SIMT: mesma instrução)
- Warp scheduler: a cada ciclo escolhe um warp pronto; troca de warp esconde a latência da memória
- `if` divergente no warp = caminhos serializados
- T4 (Turing): 40 SMs, 4 schedulers/SM, 32 warps/SM



HIERARQUIA DE MEMÓRIA (CUDA E T4)

- Modelo CUDA: privada por thread (registradores), shared por bloco, global para todos
- Shared + `__syncthreads()`: threads do bloco cooperam (tiling)
- Em GPUs de datacenter (A100/H100) a global é HBM; na T4 é GDDR6, 16 GB, 300 GB/s
- L2 (4 MB na T4) fica entre os SMs e a DRAM



TENSOR CORES

- Unidade de multiplicação de matrizes dentro do SM (desde Volta; Turing na T4)
- T4: 65 TFLOPS FP16, 130 TOPS INT8, 260 TOPS INT4 *vs.* 8,1 TFLOPS FP32 nos CUDA cores
- Só rendem com matrizes grandes (GEMM): prefill
- No decode batch=1 a conta é matriz $\times$ vetor (GEMV): tensor core fica quase ocioso
- Da GTX 980 (2014) à H100 (2022): 16 $\Rightarrow$ 132 SMs e 4,6 $\Rightarrow \sim$1000 TFLOPS, principalmente por tensor cores (CMU 15-779)

2. O QUE O BENCHMARK MEDE

A JANELA DE TEMPO DO `timed_generate`

```
cuda_sync(); start = time.perf_counter()
generated = model.generate(**inputs, ..., use_cache=True, streamer=timer)
cuda_sync(); total = time.perf_counter() - start
ttft = timer.times[0] - start          # 1o token (prefill + 1 passo)
ttlt = timer.times[-1] - start
decode_tokens_per_s = (n_new - 1) / (ttlt - ttft)
```

- Tudo é relógio da CPU: inclui GPU + Python + driver + SO
- `TokenTimer.put`: 1º callback recebe o prompt inteiro (`tolist()` + comparação) dentro do TTFT
- Cada token: `tokens.cpu()` (cópia D⇒ H) + callback Python
- Mesmo código nos 4 scripts (`qwen35`, `gemma4`, `huatuogpt`, `medgemma`)

PREFILL COMPUTE-BOUND, DECODE BANDWIDTH-BOUND

- Prefill: N tokens juntos, GEMM, muita conta por byte lido $\Rightarrow$ tensor cores
- Decode: 1 token por passo, lê todos os pesos + KV cache para produzir 1 token
- Piso de tempo por token:

$$t_{\text{piso}} \approx \frac{\text{bytes de pesos} + \text{bytes de KV}}{\text{banda}}$$

- Ex.: 4 B parâmetros em 4-bit $\approx$ 2 GB $\Rightarrow 2/300 \approx 6{,}7$ ms/token $\Rightarrow$ teto $\approx$ 150 tok/s (conta de bolso)

- Batch=1 e dependência token a token: latency-bound
- Poucos blocos por kernel, SMs ociosos, muitos kernels pequenos
- Por isso o custo fixo por kernel (launch, driver, SO) aparece no tok/s

LAUNCH OVERHEAD, CUDA GRAPHS E `torch.compile`

- Cada passo de decode = centenas de kernels (camadas $\times$ {attn, MLP, norm, dequant, LoRA})
- Cada launch custa alguns µs de CPU + driver; se o kernel dura poucos µs, a GPU espera a CPU
- CUDA Graphs: captura a sequência e relança com uma chamada; exige formas e endereços fixos
- `torch.compile(mode="reduce-overhead")` + `cache_implementation="static"` usa CUDA Graphs no Transformers
- No Windows: Inductor precisa de Triton (`triton-windows`, comunitário) $\Rightarrow$ validar antes com Unsloth/bitsandbytes/LoRA

SINCRONIZAÇÃO HOST-DEVICE E STREAMER

- `.cpu()`, `.item()`, `.tolist()`, `bool(tensor)` e `synchronize()` bloqueiam a CPU até a GPU terminar
- O `generate` já checa parada (`bool(...)`) a cada token: 1 sync/token com ou sem streamer
- Streamer adiciona: cópia D⇒H + chamada Python por token
- Cada sync = ida e volta pelo driver: é aí que o Windows pesa



azul: launch cinza: kernels
vermelho: sync + streamer (GPU ociosa)

CLOCKS (DVFS), WARM-UP E `nvidia-smi -lgc`

- A GPU sobe e desce o clock conforme carga, temperatura e limite de potência (DVFS)
- Warm-up atual = 8 tokens: curto para estabilizar clock, alocador e caches de kernel
- Proposta: 3 gerações completas (160 tokens) descartadas antes de medir
- `nvidia-smi -lgc 1590,1590` trava o clock (Linux: root; Windows: admin, depende de GPU/driver; no Colab normalmente sem permissão) $\Rightarrow$ sempre registrar clock:

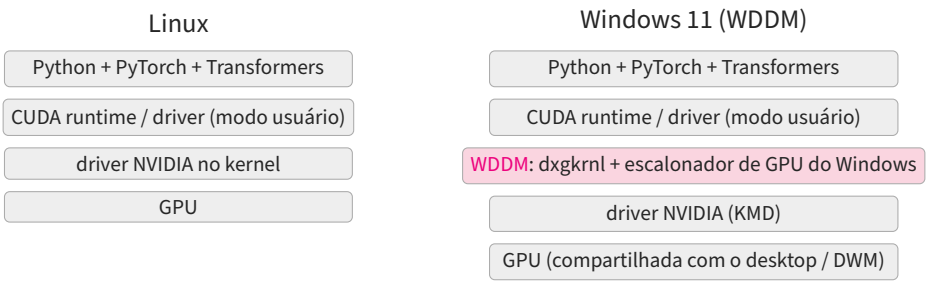
```
nvidia-smi
--query-gpu=clocks.sm,clocks.mem,temperature.gpu,power.draw,
clocks_throttle_reasons.active --format=csv -lms 200
```

QUANTIZAÇÃO 4-BIT (NF4) E DEQUANTIZAÇÃO

- `LOAD_IN_4BIT = True`: pesos em NF4 (bitsandbytes / QLoRA) $\Rightarrow$ 4$\times$ menos bytes que FP16
- Decode é bandwidth-bound $\Rightarrow$ menos bytes ajuda
- Mas cada matmul precisa dequantizar para FP16 antes/durante a conta: kernels extras
- Condição fine-tuned soma LoRA (r=8) em attn + MLP: mais kernels por token
- Diferença de tok/s base *vs.* fine-tuned mistura modelo e número de kernels: comparar com LoRA mesclado (`merge_and_unload`) ou citar a ressalva

3. OVERHEAD DO WINDOWS 11

PILHA DE SOFTWARE: LINUX *VS.* WINDOWS 11



Toda submissão e toda espera passam pela camada extra do WDDM. Em WSL2 há ainda a paravirtualização (VM $\Rightarrow$ dxgkrnl do host).

WDDM *vs.* TCC

- Windows tem 2 modelos de driver para GPU NVIDIA (guia de instalação CUDA para Windows):
  - WDDM: para GPUs com display; é o padrão em GeForce/RTX
  - TCC: só GPUs sem display (Tesla, alguns Quadro/RTX pro, GTX Titan); usa o modelo WDM
- GeForce não suporta TCC (exceto GTX Titan) $\Rightarrow$ num PC gamer o benchmark sempre roda em WDDM
- Ver/trocar: `nvidia-smi -q | findstr "Driver Model"`; `nvidia-smi -i 0 -dm TCC` (admin, sem monitor, reinicia o driver)
- TCC tira a GPU do escalonador gráfico: é o mais próximo do Linux dentro do Windows

HARDWARE-ACCELERATED GPU SCHEDULING (HAGS) E WSL2

- Sem HAGS, uma thread de alta prioridade na CPU agenda o trabalho da GPU; a Microsoft cita limites de overhead de submissão e latência até a GPU
- Com HAGS, o agendamento vai para um processador dedicado na GPU
- Ligar/desligar: Configurações ⇒ Sistema ⇒ Tela ⇒ Gráficos ⇒ Configurações padrão ⇒ “Agendamento de GPU acelerado por hardware” (reiniciar)
- NVIDIA (CUDA no WSL2): Submit assíncrono + HAGS reduziram o custo de launch; o gap que resta é em cargas pequenas e sequenciais, onde a latência de launch não se esconde
- É exatamente o perfil do decode batch=1 ⇒ o efeito aparece no tok/s, não no prefill

OUTRAS FONTES DE OVERHEAD NO WINDOWS 11

- Sysmem fallback: com VRAM cheia o driver usa RAM via PCIe sem erro $\Rightarrow$ tok/s despenca. Painel NVIDIA $\Rightarrow$ “CUDA – Sysmem Fallback Policy” $\Rightarrow$ *Prefer No Sysmem Fallback*
- Desktop na mesma GPU: DWM, navegador e vídeo dividem a GPU e a VRAM
- Energia: plano “Desempenho máximo”; Painel NVIDIA $\Rightarrow$ “Modo de gerenciamento de energia” $\Rightarrow$ *Preferir desempenho máximo*; notebook na tomada
- CPU: antivírus, indexação, updates disputam o núcleo que lança kernels
- TDR: kernel $>$ `TdrDelay` (padrão 2 s) é abortado — não muda latência, mas derruba runs longos
- Relógio não é o problema: `time.perf_counter` usa QPC (≤ 1 µs); evite `time.time()`

COMO CALCULAR O OVERHEAD: DECOMPOSIÇÃO

Por token de decode i:

$$t_i^{\text{wall}} = \underbrace{t_i^{\text{GPU}}}_{\text{kernels}} + \underbrace{t_i^{\text{launch}} + t_i^{\text{sync}}}_{\text{driver + SO}} + \underbrace{t_i^{\text{py}} + t_i^{\text{streamer}}}_{\text{Python}}$$

- t^{wall}: `perf_counter` (o que o `TokenTimer` mede)
- t^{GPU}: CUDA events ou Nsight Systems (relógio da GPU, $\sim$0,5 µs, independe do SO)
- Overhead de host por token e fração:

$$O_{\text{host}} = \overline{t^{\text{wall}}} - \overline{t^{\text{GPU}}} \qquad f_{\text{host}} = \frac{O_{\text{host}}}{\overline{t^{\text{wall}}}}$$

- Streamer: $\Delta_{\text{streamer}} = t_{\text{com}}^{\text{wall}} - t_{\text{sem}}^{\text{wall}}$ (mesmo prompt e tokens)

COMO CALCULAR O OVERHEAD DO WINDOWS (ATRIBUIÇÃO)

- Mesma GPU, mesmos pesos, prompt, `max_new_tokens`, versões e clock registrado; mude só o SO:

$$\Delta_{\text{Win}} = \tilde{t}^{\,\text{wall}}_{\text{Windows}} - \tilde{t}^{\,\text{wall}}_{\text{Linux}} \qquad \%_{\text{Win}} = \frac{\Delta_{\text{Win}}}{\tilde{t}^{\,\text{wall}}_{\text{Windows}}}$$

  ($\tilde{t}$ = mediana por token; Linux = dual boot ou live USB; WSL2 é um terceiro caso, não o baseline)

- Checagem: se t^{GPU} é igual nos dois, a diferença é host/driver, não a GPU
- Custo por kernel: $\delta_{\text{launch}} = \Delta_{\text{Win}}/K$, com K = kernels por token (contados no Nsight)
- Ablações para separar causas: HAGS on/off, desktop em outra GPU (iGPU), TCC (se a placa permitir), com/sem streamer
- Sem Linux disponível: use O_{host} e a eficiência de banda $\eta = t_{\text{piso}}/\tilde{t}^{\,\text{wall}}$

EXEMPLO NUMÉRICO (ILUSTRATIVO, NÃO MEDIDO)

	Linux	Windows 11 (WDDM)
$\tilde{t}^{\text{wall}}$ por token	30,0 ms	36,0 ms
$\tilde{t}^{\text{GPU}}$ por token (events)	27,0 ms	27,2 ms
O_{host}	3,0 ms (10%)	8,8 ms (24%)
decode tok/s	33,3	27,8

- $\Delta_{\text{Win}} = 6{,}0$ ms/token $\Rightarrow \%_{\text{Win}} = 6/36 \approx 17\%$
- GPU quase igual (27,0 *vs.* 27,2) $\Rightarrow$ a diferença é host/driver/SO
- Com $K = 600$ kernels/token: $\delta_{\text{launch}} \approx 6\,000/600 = 10$ µs a mais por kernel
- Se t^{GPU} também mudar, olhe clock/potência antes de culpar o Windows

MICROBENCHMARK: LATÊNCIA DE LAUNCH E DE SYNC

```
import time, torch
x = torch.zeros(1, device="cuda")

def launch_us(n=10000):  # só enfileira kernels minúsculos
    torch.cuda.synchronize(); t = time.perf_counter()
    for _ in range(n):
        x.add_(1)
    torch.cuda.synchronize()
    return (time.perf_counter() - t) / n * 1e6

def sync_us(n=2000):     # launch + espera, como 1 passo de decode
    torch.cuda.synchronize(); t = time.perf_counter()
    for _ in range(n):
        x.add_(1); torch.cuda.synchronize()
    return (time.perf_counter() - t) / n * 1e6

for _ in range(3):
    launch_us(1000)       # warm-up
print("launch us:", launch_us(), "| launch+sync us:", sync_us())
```

Rodar igual no Windows, WSL2 e Linux. A diferença entre SOs é o custo fixo de driver/SO por kernel e por sync.

TEMPO DE GPU NO BENCHMARK: CUDA EVENTS POR TOKEN

```
from transformers import LogitsProcessor, LogitsProcessorList

class EventTimer(LogitsProcessor):   # 1 evento por passo, sem sync
    def __init__(self):
        self.events = []
    def __call__(self, input_ids, scores):
        ev = torch.cuda.Event(enable_timing=True); ev.record()
        self.events.append(ev)
        return scores

timer = EventTimer()
start = torch.cuda.Event(enable_timing=True); end = torch.cuda.Event(enable_timing=True)
cuda_sync(); start.record(); t0 = time.perf_counter()
model.generate(**inputs, min_new_tokens=160, max_new_tokens=160, do_sample=False,
               use_cache=True, logits_processor=LogitsProcessorList([timer]))
end.record(); cuda_sync(); wall = time.perf_counter() - t0
gpu = start.elapsed_time(end) / 1000
ms = [start.elapsed_time(ev) for ev in timer.events]
print("wall:", wall, "gpu:", gpu, "O_host:", wall - gpu)
```

Não executado aqui (sem GPU). Events usam o relógio da GPU: lacunas ociosas entre kernels entram no tempo.

NSIGHT SYSTEMS NO WINDOWS: ONDE A GPU FICA PARADA

- `nsys profile -t cuda,nvtx,osrt -o win python modelos/qwen35.py`
- `nsys stats -r cuda_api_sum,cuda_gpu_kern_sum win.nsys-rep`
- K = nº de kernels / tokens gerados; soma das durações = $t^{\text{GPU busy}}$
- Fração ociosa: $1 - t^{\text{GPU busy}}/t^{\text{wall}} \Rightarrow$ alta = launch-bound (CPU/driver/SO)
- `cuda_api_sum`: tempo de `cudaLaunchKernel` e `cudaStreamSynchronize` na CPU — comparar entre Windows e Linux
- Marcar prefill e decode com `torch.cuda.nvtx.range_push/pop`

4. PROTOCOLO E CHECKLIST

PROTOCOLO DE MEDIÇÃO (WINDOWS 11)

1. Registrar: build do Windows, driver, modelo de driver (WDDM/TCC), HAGS, CUDA, PyTorch, Transformers, bitsandbytes, Unsloth, modelo e `LOAD_IN_4BIT`
2. Monitor na iGPU se possível; fechar navegador/jogos; tomada + desempenho máximo; sysmem fallback desligado
3. 1 run frio (anotar à parte) + 3 gerações completas de warm-up descartadas
4. $N \geq 20$ repetições, saída fixa (`min_new_tokens`), `do_sample=False`
5. 3 variantes: wall com streamer, wall sem streamer, CUDA events
6. Log de clock/temperatura/potência/throttle em paralelo (`nvidia-smi -lms 200`)
7. Reportar mediana, p90 e IC 95% (bootstrap); só atribuir ao Windows o que a ablação mostrar

CHECKLIST DE OTIMIZAÇÃO DO BENCHMARK

- Medir GPU com CUDA events junto do wall-clock: reportar os dois e O_{host}
- Warm-up maior: 3×160 tokens em vez de 8
- Medir sem streamer e reportar Δ_{streamer}; prompt grande pesa no TTFT pelo 1º `put`
- Fixar/registrar clocks; descartar runs com throttle
- Comparar base *vs.* fine-tuned com LoRA mesclado para não medir kernels extras
- Piloto de `torch.compile` + cache estático (CUDA Graphs) para cortar launch overhead
- No Windows: HAGS on, sysmem fallback off, desktop fora da GPU; se puder, Linux ou TCC

ONDE ACHAR SLIDES OFICIAIS E REFERÊNCIAS

- NVIDIA GTC On-Demand, série S. Jones: https://www.nvidia.com/en-us/on-demand/session/gtcfall22-a41101/ (How CUDA Programming Works), https://www.nvidia.com/en-us/on-demand/session/gtcspring23-S51210/, https://www.nvidia.com/en-us/on-demand/session/gtc24-s62401/, https://www.nvidia.com/en-us/on-demand/session/gtc25-s72897/
- OLCF/NERSC CUDA Training Series: https://www.olcf.ornl.gov/cuda-training-series/; DLI Teaching Kits: https://developer.nvidia.com/teaching-kits
- CMU 15-779 L3–4 (Z. Jia, 2025): https://www.cs.cmu.edu/~zhihaoj2/15-779/slides/03-CUDA-programming.pdf
- Turing whitepaper e T4 datasheet (nvidia.com); CUDA Best Practices: https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/
- WDDM/TCC: https://docs.nvidia.com/cuda/cuda-installation-guide-microsoft-windows/; `nvidia-smi`: https://docs.nvidia.com/deploy/nvidia-smi/
- HAGS: https://devblogs.microsoft.com/directx/hardware-accelerated-gpu-scheduling/
- CUDA no WSL2: https://docs.nvidia.com/cuda/wsl-user-guide/, https://developer.nvidia.com/blog/leveling-up-cuda-performance-on-wsl2-with-new-enhancements/
- Sysmem fallback: https://nvidia.custhelp.com/app/answers/detail/a_id/5490; TDR: https://learn.microsoft.com/en-us/windows-hardware/drivers/display/tdr-registry-keys
- QPC: https://learn.microsoft.com/en-us/windows/win32/sysinfo/acquiring-high-resolution-time-stamps; Nsight Systems: https://docs.nvidia.com/nsight-systems/UserGuide/
- PyTorch CUDA semantics: https://docs.pytorch.org/docs/stable/notes/cuda.html; HF LLM optims: https://huggingface.co/docs/transformers/llm_optims